Question:

Containerization with Docker:

Tool: Docker

Program:

- Write a Dockerfile for a Python or Node.js application.
- Build and run a Docker image.
- Push the image to Docker Hub.
- Use Docker Compose to manage multi-container applications (e.g., a web app and a database).

Procedure

1. Install Docker Desktop and ensure Docker Engine is running.
2. Create app1 as a Flask service provider application.
3. Create app2 as a service consumer application that communicates with app1.
4. Write Dockerfiles for both applications.
5. Create a docker-compose.yml file to define and run multi-container services.
6. Build Docker images using docker-compose build.
7. Run containers using docker-compose up.
8. Verify communication between containers.

Code:

Pg3/app.py:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Dockerized Flask App!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

pg3/requirements.txt

```
flask
```

pg3/Dockerfile

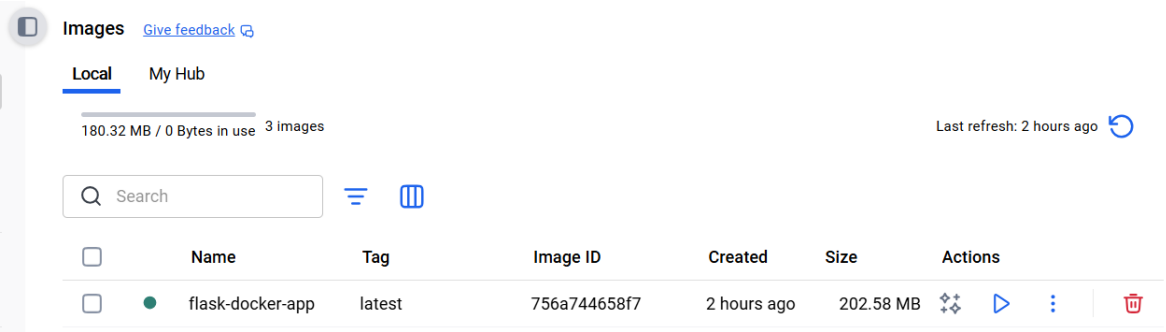
```
FROM python:3.9-slim
```

```
WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Test Data

http://localhost:5000

Hello from Dockerized Flask App!



Multi container Application:

```
pgm-3_Multicontainer/
├── app1/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── app2/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── docker-compose.yml
```

app1/app.py

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello():
    return "Hello from App 1!"

if __name__ == '__main__':
```

```
app.run(host='0.0.0.0', port=5000)
```

app1/requirements.txt

```
flask
```

app1/Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

✓ App-1 is the **service provider**

app2/app.py

```
import requests

response = requests.get("http://app1:5000/")
print("Response from App 1:", response.text)
```

app2/requirements.txt

```
requests
```

app2/Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

pgm-3_Multicontainer /docker-compose.yml

yaml

```yaml
version: "3.9"

services:
  app1:
    build: ./app1
    container_name: app1
    ports:
      - "5000:5000"
```

```
  app2:
    build: ./app2
    depends_on:
      - app1
```

Command to run:

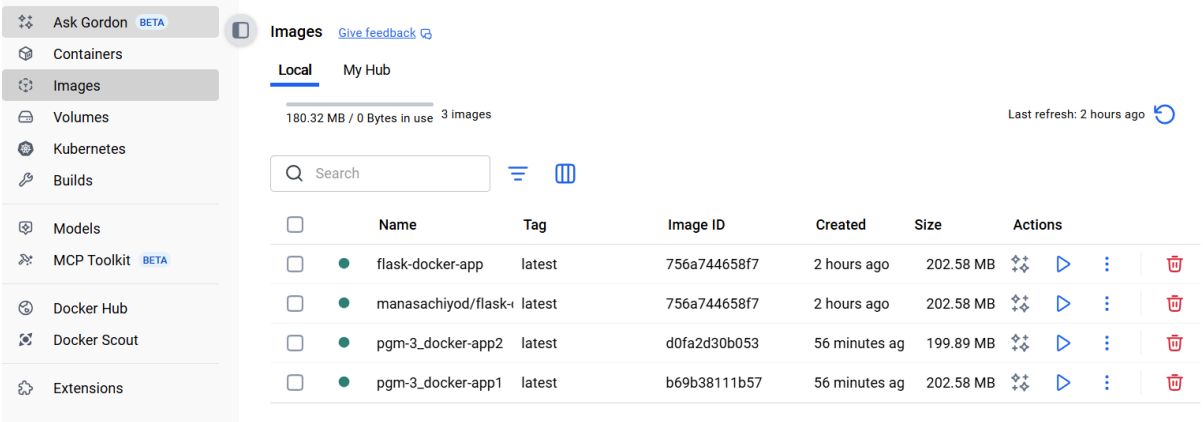
```
docker-compose up --build
```

Terminal Output

```
app1   | 172.18.0.3 - - [25/Jan/2026 14:55:43] "GET / HTTP/1.1" 200 -
app2-1  | Response from App 1: Hello from Dockerized Flask App!
app2-1 exited with code 0
app1    | 172.18.0.3 - - [25/Jan/2026 14:56:05] "GET / HTTP/1.1" 200 -
app2-1 exited with code 0
app1    | 172.18.0.1 - - [25/Jan/2026 14:56:13] "GET / HTTP/1.1" 200 -
app1    | 172.18.0.1 - - [25/Jan/2026 14:56:13] "GET /favicon.ico HTTP/1.1" 404 -
app1 exited with code 137

v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

Browser Output

Open: `http://localhost:5000`

Hello from App 1!

(paste your respective docker image)